

create_pg_ring_pattern

NAME

create_pg_ring_pattern
Creates a power ground (PG) ring pattern. The pattern created by this command is associated with a region or area of the design by using the set_pg_strategy command. The actual ring is inserted into the design by the compile_pg command.

SYNTAX

```
status create_pg_ring_pattern
pattern_name
[-nets net_list]
[-horizontal_width width_list]
[-vertical_width width_list]
[-horizontal_spacing spacing_list]
[-vertical_spacing spacing_list]
[-horizontal_layer layer]
[-vertical_layer layer]
[-side_width width_spec]
[-side_spacing spacing_spec]
[-side_layer layer_spec]
[-corner_bridge true | false | @param}]
[-track_alignment track | half_track | none |@var]
[-parameters var_list]
[-via_rule via_rule_spec]
```

Data Types

pattern_name	string
net_list	list
width_list	list
spacing_list	list
layer	string
width_spec	specification
spacing_spec	specification
layer_spec	specification
param	string
var_list	list
via_rule_spec	specification

ARGUMENTS

- pattern_name
Specifies the name of the ring pattern.
- nets net_list
Specifies power and ground net names used in this ring pattern. The net names are symbolic names and are mapped to actual net names by the set_pg_strategy command.
- The net name mapping is by position, where the first net name in the net_list list corresponds to the first net name specified by the set_pg_strategy command, and so on. This option is required when you specify a net-based via rule, as each net can have a different via requirement. Otherwise, this option is not required.
- horizontal_width width_list
Specifies the horizontal ring width. width_list specifies the list of horizontal widths, where the first width in the list is the width of the first net, and so on. The unit is um. Each value can also be specified by evaluating the variable in the -parameters option. The at character (@) is used as a prefix symbol to specify a parameter and directs the command to replace the value with the value specified by the set_pg_strategy command. If this option is not specified, the minimum width defined in the technology file is used for the horizontal width.
- vertical_width width_list
Specifies the vertical ring width. width_list specifies the list of vertical widths, where the first width in the list is the width of the first net, and so on. The unit is um. Each value can also be specified by evaluating the variable in -parameters option. The at character (@) is used as a prefix symbol to specify a parameter. If this option is not specified, the minimum width defined in the technology file is used for the vertical width.

`-horizontal_spacing spacing_list`

Specifies the spacing between horizontal ring segments. `spacing_list` specifies the list of horizontal spacing values, where the first value in the list is the spacing of the first net, and so on. The unit is `um`. Each value can also be specified by evaluating the variable in `-parameters` option. The at character (`@`) is used as a prefix symbol to specify a parameter. If this option is not specified, the minimum spacing defined in the technology file is used for horizontal spacing.

`-vertical_spacing spacing_list`

Specifies the spacing between vertical ring segments. `spacing_list` specifies the list of vertical spacing values. The first value in the list is the spacing of the first net, and so on. The unit is `um`. Each value can also be specified by evaluating the variable in `-parameters` option. The at character (`@`) is used as a prefix symbol to specify a parameter. If this option is not specified, the minimum spacing defined in the technology file is used for vertical spacing.

`-horizontal_layer layer`

Specifies the horizontal layer name. The layer name can also be specified by evaluating the variable in `-parameters` option. The at character (`@`) is used as a prefix symbol to specify a parameter. If this option is not specified, the topmost metal layers in the preferred routing direction defined in the technology file is used.

`-vertical_layer layer`

Specifies the horizontal layer name. The layer name can also be specified by evaluating the variable in `-parameters` option. The at character (`@`) is used as a prefix symbol to specify a parameter. If this option is not specified, the topmost metal layers in the preferred routing direction defined in the technology file is used.

`-side_width width_spec`

Specifies the ring segment width per side. The `width_spec` is defined as follows:

```
{{side : id_list}
 {width : width_list}}
```

where `side` is the keyword to create a side specification. `Side` contains a list of edge IDs. Each ring edge is indexed by a unique number. Edge numbering starts from 1 at the leftmost vertical edge. If there is more than one leftmost edge, the bottommost edge is the starting edge. The edge number increases by 1 as you proceed around the shape in the clockwise direction.

`width` is the keyword to create a ring width specification for that side. `width_list` specifies the width list for each net. The unit is `um`. Each value can also be specified by evaluating the variable in `-parameters` option. The at character (`@`) is used as a prefix symbol to specify a parameter.

You can create multiple specifications, where each spec is contained within a brace pair.

The `side width` specified in this option has a higher priority than the horizontal or vertical width.

`-side_spacing spacing_spec`

Specifies the ring segment spacing per side. The `spacing_spec` is defined as follows:

```
{{side : id_list}
 {spacing: spacing_list}}
```

where `side` is the keyword for side. `Side` contains a list of edge IDs. Each ring edge is indexed by a unique number. Edge numbering starts from 1 at the leftmost vertical edge. If there are more than one leftmost edges, the bottommost edge is the starting edge. The edge number increases by 1 as you proceed around the shape in the clockwise direction.

`spacing` is the keyword for ring spacing of that side. `spacing_list` specifies the spacing list between nets. The unit is defined in the technology file. Each value can also be specified by evaluating the variable in `-parameters` option. The at character (`@`) is used as a prefix symbol to specify a parameter.

You can create multiple specifications, where each spec is contained within a brace pair.

The side spacing specified in this option has a higher priority than the horizontal or vertical spacing.

`-side_layer layer_spec`

Specifies the ring segment layer per side. The `layer_spec` is defined as follows:

```
{{side : id_list}
 {layer: layer_name}}
```

where `side` is the keyword for side. `Side` contains a list of edge IDs. Each ring edge is indexed by a unique number. Edge numbering starts from 1 at the leftmost vertical edge. If there are more than one leftmost edges, the bottommost edge is the starting edge. The edge number increases by 1 as you proceed around the shape in the clockwise direction.

`layer` is the keyword for ring layer of that side. `layer_name` specifies the layer name for each net. Each value can also be specified by evaluating the variable in `-parameters` option. The at character (@) is used as a prefix symbol to specify a parameter.

You can create multiple specifications, where each spec is contained within a brace pair.

The side layer specified in this option has a higher priority than the horizontal or vertical layer.

`-corner_bridge true | false | @param`

Specifies whether the pattern creates a corner bridge. You can specify the setting by evaluating the variable `param` in the `-parameters` option. The at character (@) is used as a prefix symbol to specify a parameter. The "true" setting creates bridge connection at all ring corners and connects inner and outer rings of the same net. If this option is not specified, no bridge connection is created for the ring pattern.

`-track_alignment track | half_track | none |@var`

Specifies the track alignment option of this wire pattern. It can be specified by `track`, `half_track` or `none`. It can also be specified by evaluating the `var` variable in the `-parameters` option. The at character (@) character is used as a prefix to evaluate the variable. The default value is `none`.

`-parameters var_list`

Specifies the list of parameters for this pattern. Each parameter can be evaluated and used as the argument for other options in this command. This allows the pattern to be programmable and reused by passing different parameters in different situations. Each parameter `var` in the list `var_list` is a string. Parameter strings in `var_list` for the `-parameters` option do not contain the at character (@), but the at character is used as a prefix symbol for parameters when used with the other command options. Parameter values are specified by the `set_pg_strategy` command. See the example section for more details.

`-via_rule via_rule_spec`

Specifies the via rule for creating vias between horizontal and vertical ring segments. The via rule defines the intersection locations and the via at the locations.

There are several ways to define the intersection locations in the `via_rule_spec` specification. The syntax is defined as follows:

```
{intersection : all}      {via_def} |
{intersection : adjacent} {via_def} |
list_of_filter_rules |
```

The first way is "{intersection : all}", where `intersection` is the keyword and `all` refers to all intersections between any ring segments in different directions and different layers.

The second way is "{intersection : adjacent}", where `intersection` is the keyword, and `adjacent` refers to all intersections between any two ring segments in different directions only in adjacent layers.

The third way is to define the intersections based on two sets of shapes using `list_of_filter_rules`, which contains a list of filtering rules. Each filtering rule is specified inside a brace pair. The syntax of each filtering rule is as follows:

```
{{set_1} {filter_1}} {{set_2} {filter_2}} {via_def} |  
{intersection : undefined} {via_def}
```

where the two sets are the set of ring segments. Each set can be further filtered to obtain a subset of selected shapes. `set_1` and `set_2` are defined as follows:

```
{side : id_list}
```

where `side` is the keyword for ring side, which contains a list of edge IDs `id_list`.

The filtering operations for ring segments are defined as follows:

```
{nets : net_list}  
{layers : layer_list}  
{width : {lower_w upper_w}}
```

`nets`, `layers`, `width` keywords can be used to specify the wire net names by `net_list`, wire layer names by `layer_list` or width range by `{lower_w upper_w}`. Ring segments which satisfy the above criteria form a subset of shapes.

For those intersections which do not belong to any filtering rule, the via definition can be specified by

```
{intersection : undefined}{via_def}
```

where `intersection` is the keyword, `undefined` refers to the remaining intersections which do not belong to any filtering rules. By default, vias are not created at the intersections if they do not belong to a filtering rule, unless otherwise specified. Multiple via filtering rules, such as intersection location and via definition, can be defined and separated by braces, where each filter rule is in one brace pair.

With the intersection definition, the via specification `{via_def}` is in the following format:

```
{via_master : via_master_list | via_rule_list}
```

where `via_master` is the keyword for both via masters and via master rules. `via_master_list` specifies the list of via masters. Via masters include via contact code defined in the technology file or user-defined via cells. `via_rule_list` is a list of PG via rules. Via master rules are defined by the `set_pg_via_master_rule` command. `NIL` can be used as a keyword to indicate that no via is created. `default` can be used to apply the default vias in the specified location. For each specified intersection, vias are created based on the specified via masters or via master rules. If `NIL` is not specified in the list, the default contact code is used to complete a stacked via or to replace a via when a DRC error occurs.

By default, the default via defined in the technology file is created at each intersection of orthogonal ring segments, where the ring segments are on different layers.

DESCRIPTION

Creates a power ground (PG) ring pattern. The pattern created by this command is associated with a region or area of the design by using the `set_pg_strategy` command, and is inserted into the design by the `compile_pg` command.

The ring pattern specifies the horizontal and vertical ring width values, layer names, spacing values and corner bridge options. The values can be either fixed values or set by using parameters. Via rules between horizontal and vertical ring segments can also be specified.

The ring contour shape, offset to the ring contour and vias between this ring pattern and any other shapes can be specified by `set_pg_strategy` command when associating this pattern to a routing area in the design.

EXAMPLES

The following example creates a ring pattern `ring1` for layers `M3` and

M4. The spacing values are 1 and 2, the width is 5 for horizontal and vertical segments except for side 1 with width 6, the corner bridge option is defined by parameter @flag, and the via master rule via34_2x2, i.e. between layer M3 and M4 with array size 2 by 2.

```
prompt> create_pg_ring_pattern ring1 \  
-parameters {flag} \  
-horizontal_layer M3 -vertical_layer M4 \  
-horizontal_width 5 -vertical_width 5 \  
-horizontal_spacing 1 -vertical_spacing 2 \  
-side_width {{side: 1} {width: 6}} \  
-corner_bridge @flag \  
-via_rule {{intersection: all} {via_master: VIA34}}
```

The following example creates a ring pattern ring2, where the width list {1 2 3} is applied to sides {1 3} and width list {3 4 5} is applied to sides {2 4}. For sides 1 and 3, the innermost ring width is 1, middle ring width is 2 and outermost ring width is 3. For sides 2 and 4, the innermost ring width is 3, middle ring width is 4 and outermost ring width is 5.

```
prompt> create_pg_ring_pattern ring2 \  
-side_width { \  
  {{side: 1 3} {width: 1 2 3}} \  
  {{side: 2 4} {width: 3 4 5}} \  
}
```

The following example creates a ring pattern ring3, where the width list {1 2 3} is applied to sides {1 2 3 4}. For sides 1, 2, 3 and 4, the innermost ring width is 1, the middle ring width is 2 and the outermost ring width is 3.

```
prompt> create_pg_ring_pattern ring3 \  
-side_width { \  
  {side: 1 2 3 4} {width: 1 2 3} \  
}
```

MORE EXAMPLES

For more example, please start GUI and invoke the following command.
gui_show_task_assistant -task "Design Planning:PG Planning->Examples->Overview".

SEE ALSO

[compile_pg\(2\)](#)
[create_pg_composite_pattern\(2\)](#)
[create_pg_macro_conn_pattern\(2\)](#)
[create_pg_std_cell_conn_pattern\(2\)](#)
[create_pg_wire_pattern\(2\)](#)
[remove_pg_patterns\(2\)](#)
[report_pg_patterns\(2\)](#)
[report_pg_strategies\(2\)](#)
[set_pg_strategy\(2\)](#)
[set_pg_via_master_rule\(2\)](#)

Version V-2023.12-SP5-6

Copyright (c) 2025 Synopsys, Inc. All rights reserved.